

Post-Quantum Aliro: Integrating NIST Post-Quantum Cryptography Standards into NFC Access Control

Anonymous

Anonymous Institution

Abstract. The widespread deployment of quantum computers threatens the long-term confidentiality and authenticity guarantees of classical public-key cryptography, placing security-critical infrastructure such as NFC-based physical access control at risk. Aliro 1.0, the CSA standard for NFC/BLE/UWB access control, relies exclusively on elliptic-curve primitives—ECDH key agreement and ECDSA signatures over P-256—that are vulnerable to Shor’s algorithm. We present PQ-Aliro, a post-quantum adaptation of Aliro that replaces ECDH with ML-KEM-768 (CRYSTALS-Kyber) and ECDSA with ML-DSA-65 (CRYSTALS-Dilithium) while preserving the original three-round structure (AUTH0 / LOADCERT / AUTH1), the HKDF-based session-key derivation (HMAC-SHA256 + HKDF-Expand), and AES-256-GCM message encryption. We prove, within the AKE security model of Schäge et al. [30] (which extends the Bellare–Rogaway framework [2] with original-key partnering and responder-identity privacy), that PQ-Aliro achieves mutual authentication, forward secrecy (under long-term key corruption), and identity privacy under the IND-1CCA security of ML-KEM-768, the EUF-CMA security of ML-DSA-65, the PRF security of HKDF, and the AEAD security of AES-256-GCM. A full prototype is implemented on Android HCE and a PC NFC reader using BouncyCastle 1.78 BCPQC; an end-to-end test suite of 20 assertions covering 9 PQC configurations passes without error. The default configuration incurs approximately 16.4 KB across 75 APDUs—roughly 14 times the ECC baseline of 1.2 KB—while a key-slot caching optimisation reduces this to 11.2 KB, and the lightest viable configuration (D2×K512) achieves 11.9 KB, making deployment over a single ISO 14443 NFC channel practical.

1 Introduction

The advent of large-scale quantum computing poses a fundamental threat to the public-key cryptography underpinning today’s digital infrastructure. Shor’s algorithm [32] can factor integers and compute discrete logarithms in polynomial time, rendering RSA and elliptic-curve cryptography (ECC) insecure, while Grover’s algorithm [15] halves the effective security of symmetric primitives. Although the timeline for cryptographically relevant quantum computers remains uncertain, the strategic risk is real: adversaries may harvest ciphertext today

and decrypt it once quantum capability matures—a threat model known as “harvest now, decrypt later.” In response, the National Institute of Standards and Technology (NIST) has finalised two post-quantum cryptography (PQC) standards: FIPS 203 (ML-KEM, based on CRYSTALS-Kyber) [25] for key encapsulation, and FIPS 204 (ML-DSA, based on CRYSTALS-Dilithium) [26] for digital signatures. NIST IR 8547 [27] further mandates that federal agencies complete migration away from RSA and ECC by 2035. Physical access control systems are a particularly sensitive target: NFC-based door locks and vehicle entry systems guard critical infrastructure and personal safety, yet virtually all deployed protocols rely on ECC. Transitioning these systems to post-quantum security before the quantum threat materialises is therefore an urgent engineering challenge. Beyond passive harvesters, NFC access-control protocols face active adversaries capable of man-in-the-middle, replay, and relay attacks; our formal model captures the first two, while relay is mitigated by NFC’s physical proximity assumption.

Aliro 1.0 [6], published by the Connectivity Standards Alliance (CSA), is a recently standardised multi-modal access-control protocol supporting NFC, BLE, and UWB bearers. Its cryptographic core is built on ECC: an ECDH key-exchange over P-256 for session-key establishment and ECDSA for certificate-based entity authentication. Similar designs appear in the automotive domain through CCC Digital Key 3.0 [5], which likewise relies on ECDH/ECDSA for secure vehicle-phone pairing. Integrating ML-KEM and ML-DSA into such protocols is non-trivial for three interconnected reasons. First, object-size inflation: an ML-DSA-65 public key occupies 1,952 bytes and a signature 3,309 bytes, compared with 65 bytes and approximately 72 bytes for ECDSA on P-256—an increase of roughly $30\times$ and $46\times$, respectively. An ML-KEM-768 ciphertext is 1,088 bytes, versus 65 bytes for an ephemeral ECDH public key. Second, NFC channel constraints: ISO/IEC 14443 [17] ISO-DEP frames carry at most 255 bytes of application data per APDU, and the physical-layer bit rate ranges from 106 to 848 kbit/s. Even at 424 kbit/s—the rate typically negotiated by Android HCE—a single PQ-Aliro transaction under the default ML-DSA-65 $\times$ ML-KEM-768 configuration transfers approximately 16.4 KB across approximately 75 APDUs, roughly $14\times$ the approximately 1.2 KB of a classical ECC transaction. Third, platform limitations: Android’s `KeyStore` API exposes no native support for lattice-based algorithms, requiring the implementation to manage PQC key material entirely in software using third-party libraries such as BouncyCastle.

With ML-KEM and ML-DSA finalised as FIPS 203/204 in 2024 and BouncyCastle 1.78 BCPQC providing production-grade implementations, the prerequisites for a standards-compliant PQC upgrade of Aliro are now in place. This paper presents **PQ-Aliro**, which, to the best of our knowledge, is the first PQC upgrade of the Aliro NFC expedited flow (see §2). Our contributions are as follows:

1. **Protocol design and implementation.** We specify a PQC instantiation of the Aliro Expedited flow that replaces ECDH with ML-KEM-768 key encapsulation and ECDSA with ML-DSA-65 signatures, preserving the three-

round AUTH0/LOADCERT/AUTH1 message structure. We provide a full open-source implementation: an Android HCE user-device app and a PC NFC reader, both using BouncyCastle 1.78 BCPQC as the cryptographic back-end. We restrict our implementation to the NFC bearer; BLE and UWB transports and Secure Element offload are out of scope.

2. **APDU-level overhead analysis.** We conduct an exhaustive byte-accurate analysis of communication overhead for all nine PQC parameter combinations: Dilithium $\{2, 3, 5\} \times$ Kyber $\{512, 768, 1024\}$. Overhead ranges from approximately 11.9 KB ($D2 \times K512$) to approximately 23.2 KB ($D5 \times K1024$), compared to approximately 1.2 KB for the classical ECC instantiation. The default ML-DSA-65 $\times$ ML-KEM-768 configuration incurs approximately 16.4 KB and approximately 75 APDUs per transaction; the lightest ML-DSA-44 $\times$ ML-KEM-512 configuration reduces this to approximately 11.9 KB and approximately 55 APDUs. We additionally show that a key-slot optimisation—pre-caching the reader certificate on the user device—eliminates the LOADCERT round and saves approximately 5–7 KB. To our knowledge, no prior work provides a complete, APDU-granular communication-overhead analysis for a PQC-upgraded NFC access-control protocol.
3. **Formal security analysis.** Building on the AKE security model of Schäge et al. [30] (which extends Bellare–Rogaway [2] with original-key partnering and responder-identity privacy) and the provable security of ML-KEM [3] and ML-DSA [8], we prove, instantiating the Schäge et al. framework with PQC primitives, that PQ-Aliro achieves mutual authentication upon AUTH1 verification, forward secrecy (under long-term key corruption) after session completion, and reader-identity privacy against passive eavesdroppers. The security bound reduces to the IND-1CCA security of ML-KEM-768, the EUF-CMA security of ML-DSA-65, the PRF security of HKDF, and the AEAD security of AES-256-GCM (see §6–7 for the full model and proof). Our model does not capture side-channel attacks, ephemeral-state compromise, or relay attacks; the latter is mitigated at the physical layer by NFC’s inherent ~ 10 cm proximity limit.
4. **Engineering insights for constrained Android HCE.** We identify and resolve two Android-specific hazards invisible at the protocol level: Dilithium private-key exposure through JVM garbage-collection non-determinism (with a typical GC pause window of 50–200 ms, addressed by named-process isolation with immediate self-termination), and HCE main-thread deadlock from blocking ML-DSA signing calls (which triggers an Application Not Responding (ANR) event after 5 s, addressed by asynchronous dispatch). No prior PQC-over-HCE implementation has documented either hazard. These hazards and their mitigations apply to any Android HCE/HAL scenario scheduling long-latency PQC private-key operations, including CCC Digital Key, PQC X.509 certificate verification, and PQC-FIDO2.

Contributions 1 and 2 directly address the first two challenges (object-size inflation and NFC channel constraints); Contribution 3 establishes the formal se-

curity guarantees; Contribution 4 resolves the platform-specific implementation hazards.

Contribution 3 is not a formality: replacing ECDH/ECDSA with KEM/signature alters the authentication structure from explicit (signature-on-transcript) to KEM-implicit, necessitating a fresh proof that the implicit-authenticated session key still satisfies mutual authentication under the Schäge et al. model.

The remainder is organised as follows: §2 surveys related work; §3 introduces preliminaries; §4 presents the protocol; §5–6 establish security; §7–8 describe the implementation and evaluation; §9 discusses optimisations and limitations; §10 concludes.

2 Related Work

2.1 Post-Quantum Key Exchange and Authentication

The ML-KEM and ML-DSA algorithms used in PQ-Aliro are formally described in Section 3; here we focus on existing AKE constructions built from lattice primitives.

The standardisation of post-quantum cryptographic primitives has spurred a growing body of work on constructing efficient and provably secure authenticated key exchange (AKE) protocols from lattice-based building blocks.

KEMTLS [31] is a landmark engineering contribution that replaces the signature-based handshake of TLS 1.3 with KEM-based mutual authentication, eliminating handshake signatures from the critical path entirely. PQ-Aliro takes a complementary but distinct approach: rather than redesigning the handshake, it preserves the Aliro APDU command structure and substitutes only the underlying primitives, ensuring wire-level structural compatibility with existing access-control infrastructure. Crucially, KEMTLS targets TLS record-layer semantics over TCP/IP, where payload fragmentation is handled transparently by the transport layer; PQ-Aliro must instead work within the hard 255-byte ISO-DEP APDU boundary of the NFC physical layer, making byte-level communication overhead a primary design constraint rather than an afterthought.

On the theoretical side, Fujioka et al. [12] construct one-pass AKE protocols from one-way CCA-secure KEMs, establishing a principled framework for building AKE security from KEM security in the standard model. PQ-Aliro’s authentication structure follows a similar spirit: the ML-KEM encapsulation provides session-key establishment, while ML-DSA certificates bind ephemeral keys to long-term identities. Related lattice-based AKE constructions with tight security reductions, such as the work of Pan et al. [28], confirm that provably secure, tightly-reduced lattice AKE is achievable, reinforcing the theoretical foundations on which PQ-Aliro rests. Earlier work [11] demonstrated strongly secure AKE from lattice assumptions in the PKC setting, providing further evidence of the maturity of this research direction. These theoretical results, however, do not directly address the byte-level constraints that motivate the design of PQ-Aliro, which we examine in the context of AKE security models next.

2.2 AKE Security Models

The security of authenticated key exchange protocols is formalised through a hierarchy of increasingly powerful adversarial models. The foundational Bellare–Rogaway (BR93) model [2] establishes the core AKE framework: it defines session partnering via matching conversations, formalises adversarial queries including long-term-key corruption, and captures session-key indistinguishability and entity authentication. Canetti and Krawczyk [4] extended this framework with the CK model, which additionally captures leakage of session-specific internal state and allows for more refined notions of forward secrecy. The eCK model of LaMacchia, Lauter, and Mityagin [21] pushes adversarial power further, simultaneously exposing long-term keys and ephemeral randomness to the adversary, and is widely regarded as one of the strongest standard AKE security definitions.

PQ-Aliro’s security analysis is cast in a BR-style model augmented with an identity-privacy experiment. The privacy dimension is motivated by the work of Lyu et al. [23], who formalise privacy-preserving AKE in the standard model and prove that user identities can be kept unlinkable across sessions even against an active adversary. In the NFC access-control setting, tracking resilience is a practical requirement: a passive eavesdropper recording NFC transactions must not be able to link repeated accesses to a single credential holder. Our identity-privacy experiment, which tests whether an adversary can distinguish between two target users based on observed APDU transcripts, follows the definitional approach of [23] adapted to the two-party door-access setting. The TLS 1.3 cryptographic analysis of Dowling et al. [7] provides a further point of comparison for multi-stage AKE models, although its record-layer focus is orthogonal to our APDU-layer concerns. We adopt the framework of Schäge et al. [30] because, to the best of our knowledge, it is the only BR-style AKE model that simultaneously captures responder-identity privacy (tracking resilience) and original-key partnering—both essential properties for the Aliro access-control setting. The eCK model [21] provides stronger guarantees against ephemeral-state leakage but does not address identity privacy; Lyu et al. [23] model identity privacy but assume a single-session setting incompatible with multi-user Aliro deployments. This emphasis on APDU-layer constraints naturally leads to the question of how PQC primitives perform on the constrained platforms typical of NFC access control.

2.3 PQC on Constrained Platforms

Deploying post-quantum cryptography in resource- or bandwidth-constrained environments remains an active area of research. Lima et al. [34] evaluate the performance of Kyber KEM in a native Android mobile application, reporting that encapsulation and decapsulation are fast enough for interactive use on commodity smartphones. Their findings provide direct empirical support for the feasibility of mobile-side PQC in PQ-Aliro’s Android HCE implementation. Post-quantum WireGuard [16] integrates lattice-based KEMs into the WireGuard VPN protocol, demonstrating that the handshake overhead of NIST-round-3

KEMs can be absorbed by a constrained network protocol without breaking functional compatibility. Unlike WireGuard’s 1,232-byte UDP payload budget [16], PQ-Aliro must operate within a 255-byte ISO-DEP APDU limit [17], making byte-level fragmentation a first-class design constraint rather than a post-hoc optimisation.

The above results focus on IP-based transports or general mobile computing contexts. Despite these results, a significant gap remains. Existing work addresses PQC integration in TLS and VPN-class protocols, where message fragmentation is handled by TCP or UDP at the transport layer and byte-level overhead does not impose hard per-message limits. The NFC context is fundamentally different: ISO/IEC 14443 ISO-DEP APDUs carry at most 255 bytes of application data, and the 424 kbit/s physical-layer bit rate turns each kilobyte of overhead into measurable latency. To our knowledge, no prior work provides a complete, APDU-granular communication-overhead analysis for a PQC-upgraded NFC access-control protocol.

On the standards side, neither Aliro 1.0 [6] nor CCC Digital Key 3.0 [5]—the two leading specifications for smartphone-based digital access control—provides a post-quantum variant, a PQC migration roadmap, or a formal security proof for their AKE core. PQ-Aliro addresses all three dimensions—wire compatibility, byte-accurate overhead characterisation, and formal security proof—simultaneously: it delivers a wire-compatible PQC upgrade, a byte-accurate overhead characterisation across all nine ML-KEM/ML-DSA parameter combinations, and a formal security proof establishing mutual authentication, forward secrecy (under long-term key corruption), and identity privacy.

3 Preliminaries

3.1 The Aliro Access Control Protocol

Aliro 1.0 [6] is a unified access-control specification published by the Connectivity Standards Alliance (CSA) in 2026. It defines a credential-based authentication framework for NFC, BLE, and UWB-capable devices and is designed to replace proprietary card-emulation schemes with a single interoperable protocol. In the NFC context two principals take part in every transaction: the User Device (UD), typically a smartphone running Android Host Card Emulation (HCE), and the Reader, a door controller or turnstile that initiates the exchange.

Expedited Phase protocol flow. Authentication proceeds in four logical messages over the ISO/IEC 14443 Type A/B contactless channel:

1. **SELECT (AID = F0 10 20 30 40 50):** The Reader selects the Aliro applet on the UD. No payload is exchanged at this step; it serves solely to activate the HCE service.
2. **AUTH0 (INS = 0x80), Reader → UD:** The Reader transmits a fresh ephemeral public key together with a **transaction-identifier** (16 B) and

a **reader-identifier** (32 B). In the classical ECC instantiation this is a P-256 ECDH ephemeral key; in PQ-Aliro it is an ML-KEM-768 ephemeral public key. The UD responds with the key-encapsulation ciphertext (or ECDH public share) and a 32-byte noise contribution. Both parties can now derive a shared session key via HKDF.

3. **LOADCERT** (INS = 0xD1), **Reader** $\rightarrow$ **UD**: The Reader delivers its long-term certificate in Profile0000 DER format. This certificate carries the Reader’s long-term public key and the issuer’s signature over it. Upon successful verification the UD returns a two-byte status word 0x9000.
4. **AUTH1** (INS = 0x81), **Reader** $\rightarrow$ **UD**: The Reader signs the session transcript and sends the signature to the UD. After verifying the transcript signature the UD returns an AEAD-encrypted payload containing its own long-term public key, its transcript signature, and a **signaling-bitmap** (2 B), authenticated with AES-GCM using the previously derived session key.

NFC physical-layer constraints. ISO/IEC 14443 imposes strict limits on APDU payload sizes: the Reader side chunks application data into frames of at most 200 bytes, while the UD side responds in chunks of at most 240 bytes. Fields longer than 255 bytes are encoded with BER-TLV long-form length (0x82 followed by a two-byte big-endian length), consuming an extra 4 bytes of framing overhead per field. Each command APDU adds a 6-byte header (CLA, INS, P1, P2, Lc, Le), and each HCE response appends a 2-byte status word.

In the original ECC instantiation (P-256 ECDH + ECDSA) the total application-layer payload per transaction amounts to roughly 1.2 KB, typically completing in a handful of APDU round trips. As Shor’s algorithm renders elliptic-curve discrete-log problems tractable on a sufficiently large quantum computer, this ECC-based design must be migrated to post-quantum primitives — a transition that dramatically increases the cryptographic object sizes and, consequently, the NFC communication overhead.

3.2 ML-KEM and ML-DSA

ML-KEM (Kyber). ML-KEM was originally proposed as CRYSTALS-Kyber [3] and was standardised by NIST as FIPS 203 [25]. It is a key-encapsulation mechanism (KEM) whose security reduces to the hardness of the Module Learning With Errors (Module-LWE) problem. The scheme provides a triple (Gen, Encap, Decap): the receiver generates a key pair $(pk, sk) \leftarrow \text{Gen}()$; the sender produces a ciphertext and shared secret $(c, K) \leftarrow \text{Encap}(pk)$; the receiver recovers $K \leftarrow \text{Decap}(sk, c)$. ML-KEM achieves IND-CCA2 security in the random-oracle model via the Fujisaki–Okamoto transform [13]. Three parameter sets target NIST security levels 1, 3, and 5: ML-KEM-512, ML-KEM-768, and ML-KEM-1024, respectively. PQ-Aliro adopts **ML-KEM-768** (Level 3) by default, yielding a public key of 1184 bytes, a ciphertext of 1088 bytes, and a 32-byte shared secret. This KEM replaces the P-256 ECDH ephemeral exchange in the AUTH0 step.

Table 1. Cryptographic object sizes: ECC P-256 vs. ML-KEM-768 vs. ML-DSA-65

Object	ECC P-256	ML-KEM-768	ML-DSA-65
Public key	65 B	1,184 B	1,952 B
Ciphertext / Signature	72 B	1,088 B	3,309 B
Shared secret	32 B	32 B	—
NIST security level	—	Level 3	Level 3

ML-DSA (Dilithium). ML-DSA was originally proposed as CRYSTALS-Dilithium [8] and was standardised as FIPS 204 [26]. Its security relies on the hardness of Module-LWE and Module Short Integer Solution (Module-SIS), and the scheme achieves EUF-CMA security. Three parameter sets cover NIST security levels 2, 3, and 5: ML-DSA-44, ML-DSA-65, and ML-DSA-87. PQ-Aliro adopts **ML-DSA-65** (Level 3) by default, producing a public key of 1 952 bytes and a signature of 3 309 bytes. ML-DSA replaces ECDSA in two roles: the Reader embeds its long-term ML-DSA public key in the Profile0000 certificate transmitted during LOADCERT, and signs the session transcript in AUTH1; the UD likewise signs the transcript with its own ML-DSA key and returns the signature encrypted inside the AUTH1 response.

Table 1 summarises the size difference between classical P-256 objects and their post-quantum counterparts. Public keys and signatures grow by roughly 18–46 $\times$, which is the primary driver of the $\sim 13\times$ increase in total NFC traffic observed in PQ-Aliro relative to the original ECC design.

3.3 Cryptographic Definitions

Notation. We write $y \leftarrow \$ \mathcal{Y}$ to indicate that y is chosen uniformly at random from the set $\mathcal{Y}$. The notation $x := z$ means that x is assigned the value z . For a randomized algorithm f , $y \leftarrow \$ f(x)$ denotes a fresh random output obtained by executing f on input x , while $y \leftarrow f(x; r)$ denotes the deterministic evaluation of f with explicit randomness r . For a positive integer N , $[N]$ stands for the set $\{1, 2, 3, \dots, N\}$. Let λ be the security parameter; $\text{negl}(\lambda)$ denotes a negligible function in λ , and $p(\lambda)$ an arbitrary polynomial. For all polynomials p and sufficiently large λ , $\text{negl}(\lambda) < 1/p(\lambda)$.

Collision-Resistant Hash Functions

Definition 1 (Collision-Resistant Hash Function). *A hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is collision-resistant if for every PPT adversary $\mathcal{A}$,*

$$\text{Adv}_H^{\text{coll}}(\mathcal{A}, \lambda) := \Pr[(x, x') \leftarrow \mathcal{A}(1^\lambda) : x \neq x' \wedge H(x) = H(x')] \leq \text{negl}(\lambda).$$

In our protocol, H is modelled as a collision-resistant hash function; the transcript $\text{trans} = H(pk_e \| c_e)$ inherits this property.

Exp IND-1CCA($\mathcal{A}, \lambda, \text{KEM}$)	$\mathcal{O}(\text{ct})$
1 : $\text{pp} \leftarrow \text{KEM.Setup}(1^\lambda)$	1 : if $\text{ct} \neq \text{ct}^* \wedge \text{first call}$
2 : $(\text{pk}^*, \text{sk}^*) \leftarrow \text{KEM.Gen}(\text{pp})$	2 : return $\text{KEM.Decap}(\text{sk}^*, \text{ct})$
3 : $b \leftarrow \{0, 1\}$	3 : else
4 : $(\text{ss}_0, \text{ct}^*) \leftarrow \text{KEM.Encap}(\text{pk}^*)$	4 : return $\perp$
5 : $\text{ss}_1 \leftarrow \mathcal{K}$	
6 : $b' \leftarrow \mathcal{A}^\mathcal{O}(\text{pk}^*, \text{ct}^*, \text{ss}_b)$	
7 : return $b' = b$	

Fig. 1. The IND-1CCA game for $\text{KEM} = (\text{Setup}, \text{Gen}, \text{Encap}, \text{Decap})$.

Key Encapsulation Mechanism A key encapsulation mechanism (KEM) [33] is defined by the following four algorithms:

- $\text{Setup}(1^\lambda)$: on input the security parameter λ , produces public parameters pp that specify the key space $\mathcal{K}$, public key space $\mathcal{PK}$, secret key space $\mathcal{SK}$, and ciphertext space $\mathcal{C}$.
- $\text{Gen}(\text{pp})$: takes the public parameters pp and outputs a fresh key pair (pk, sk) .
- $\text{Encap}(pk)$: given a public key pk , generates a ciphertext c together with an encapsulated key K .
- $\text{Decap}(sk, c)$: using the secret key sk and a ciphertext c , recovers the key K .

Correctness. For every $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, every $(pk, sk) \leftarrow \text{Gen}(\text{pp})$, and every $(c, K) \leftarrow \text{Encap}(pk)$, it must hold that $\Pr[\text{Decap}(sk, c) \neq K] \leq \text{negl}(\lambda)$.

Remark on public parameters. For conciseness, we omit the **Setup** algorithm and public parameters pp in the sequel, as ML-KEM embeds these implicitly in the security parameter.

Definition 2 (IND-1CCA Security for KEMs). *The IND-1CCA experiment for a KEM is depicted in Figure 1. The advantage of an adversary $\mathcal{A}$ against the IND-1CCA security of KEM is defined as*

$$\text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{A}, \lambda) = \left| \Pr[\text{IND-1CCA}(\mathcal{A}, \lambda, \text{KEM}) = 1] - \frac{1}{2} \right|.$$

Remark 1. *ML-KEM-768 (CRYSTALS-Kyber) [25] achieves IND-CCA2 security in the random-oracle model via the Fujisaki–Okamoto transform [13]. Since IND-CCA2 implies IND-1CCA, the proof in Section 6 applies directly when instantiated with ML-KEM-768.*

Digital Signature Schemes We adopt the standard notion of a digital signature scheme [14].

Definition 3 (Digital Signature Scheme). *A digital signature scheme for a message space $\mathcal{M}$ is a triple of algorithms $\text{SIG} = (\text{SIG.Gen}, \text{SIG.Sign}, \text{SIG.Verif})$ where*

1. $\text{SIG.Gen}(1^\lambda)$ is the randomized key-generation algorithm that, on input the security parameter λ , outputs a public verification key pk and a secret signing key sk .
2. $\text{SIG.Sign}(sk, m)$ is the randomized signing algorithm that, given the signing key sk and a message $m \in \mathcal{M}$, produces a signature σ .
3. $\text{SIG.Verif}(pk, m, \sigma)$ is the deterministic verification algorithm that returns either 0 (invalid) or 1 (valid).

Correctness. A signature scheme SIG is **correct** if for any message $m \in \mathcal{M}$ and for every key pair (pk, sk) in the support of $\text{SIG.Gen}(1^\lambda)$, it holds that $\text{SIG.Verif}(pk, m, \text{SIG.Sign}(sk, m)) = 1$.

Existential Unforgeability under Chosen-Message Attack. The standard security requirement for digital signatures is existential unforgeability under an adaptive chosen-message attack (EUF-CMA) [14].

Definition 4 (EUF-CMA Security). *Let SIG be a digital signature scheme (Definition 3). The experiment $\text{EUF-CMA}(\mathcal{A}, \lambda, \text{SIG})$ proceeds as follows:*

1. *The challenger generates $(pk, sk) \leftarrow \text{SIG.Gen}(1^\lambda)$, initialises $Q_{\text{Sign}} := \emptyset$, and gives pk to $\mathcal{A}$.*
2. *$\mathcal{A}$ adaptively issues signature queries; for each query on m the challenger returns $\sigma \leftarrow \text{SIG.Sign}(sk, m)$ and adds m to Q_{Sign} .*
3. *$\mathcal{A}$ outputs (m^*, σ^*) . The experiment outputs 1 iff $\text{SIG.Verif}(pk, m^*, \sigma^*) = 1$ and $m^* \notin Q_{\text{Sign}}$.*

The advantage is $\text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{A}, \lambda) := \Pr[\text{EUF-CMA}(\mathcal{A}, \lambda, \text{SIG}) = 1]$.

Key Derivation Functions Key derivation employs the HMAC-based Key Derivation Function (HKDF) [18]. HKDF follows the extract-then-expand paradigm, using HMAC [20] as its building block. HKDF comprises two procedures:

- $\text{HKDF.Extract}(salt, IKM)$: extracts a pseudorandom key from input keying material using an optional salt, as $\text{HMAC.Mac}(salt, IKM)$.
- $\text{HKDF.Expand}(PRK, info, L)$: expands the pseudorandom key into L bits using context string $info$ [7].

Definition 5 (Pseudorandom Function (PRF)). *A pseudorandom function $\text{PRF} : \mathcal{K}_{\text{PRF}} \times \mathcal{L}_{\text{PRF}} \rightarrow \{0, 1\}^\lambda$ is secure if for every PPT adversary $\mathcal{A}$,*

$$\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{A}, \lambda) = \left| \Pr \left[k \leftarrow \mathcal{K}_{\text{PRF}} : \mathcal{A}^{\text{PRF}(k, \cdot)} = 1 \right] - \Pr \left[\mathcal{A}^{R(\cdot)} = 1 \right] \right| \leq \text{negl}(\lambda),$$

where $R : \mathcal{L}_{\text{PRF}} \rightarrow \{0, 1\}^\lambda$ is a uniformly random function.

In our protocol, HKDF.Extract and HKDF.Expand [18] are each modelled as a PRF instance; the input domain $\mathcal{L}_{\text{PRF}}$ corresponds to the transcript space, and the output is the session key material.

Authenticated Encryption We model a symmetric authenticated encryption with associated data (AEAD) scheme as a pair $\text{AEAD} = (\text{Enc}, \text{Dec})$ over a key space $\mathcal{K}_{\text{AEAD}}$ and message space $\mathcal{M}_{\text{AEAD}}$, following the AE security definition of Rogaway and Shrimpton [29].

Game $\text{AEAD-sec}(\mathcal{A}, \lambda, \text{AEAD})$: The game proceeds in three phases: (1) **Setup phase**: the challenger samples a fresh key $k \leftarrow \mathcal{K}_{\text{AEAD}}$ and a challenge bit $b \leftarrow \{0, 1\}$; (2) **Encryption oracle**: on each query (m_0, m_1, ad) , the challenger returns $\text{Enc}(k, m_b, ad)$; (3) **Decryption oracle**: on each query (c, ad) , the challenger returns $\text{Dec}(k, c, ad)$ unless c is a challenge ciphertext, in which case $\perp$ is returned. The advantage of $\mathcal{A}$ in the above game is:

Definition 6 (Secure AEAD). *The AEAD scheme AEAD is secure if for every PPT adversary $\mathcal{A}$ interacting with encryption and decryption oracles (where the decryption oracle refuses to decrypt challenge ciphertexts),*

$$\text{Adv}_{\text{AEAD}}^{\text{AEAD-sec}}(\mathcal{A}, \lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

The five primitive definitions above—collision-resistant hash (Definition 1), IND-1CCA KEM (Definition 2), EUF-CMA signature (Definition 4), PRF (Definition 5), and secure AEAD (Definition 6)—serve as the building blocks for the security model (§ 5) and the formal security proof (§ 6). ML-KEM-768 satisfies IND-1CCA (as noted in the remark above) and ML-DSA-65 satisfies EUF-CMA [8], which is precisely what the game-hopping reduction in § 6 requires.

4 Protocol Description

Generic Construction. The PQ-Aliro protocol is shown in Fig. 2. There are two users: Reader and User Device. Each party holds a pair of long-term keys. As is noted by the Aliro 1.0 specification, “A Reader SHALL have a key pair (reader-PubK/reader-PrivK) and MAY have a certificate (reader-Cert) containing the Reader public key. The Reader uses this key pair to authenticate itself to the User Device. . . . A User Device SHALL have an **Access Credential** with a key pair (credential-PubK/credential-PrivK) that is unique to the User Device. If an Access Document is present, it SHALL contain the public key of the Access Credential.”

The PQ-Aliro protocol runs as follows. Firstly, the Reader uses KEM.Gen to generate a pair of ephemeral keys (pk_e, sk_e) .

For expository clarity, Figure 2 presents the abstract protocol specification using generic KEM and SIG notation; the concrete ML-KEM-768 and ML-DSA-65 instantiation, APDU-layer fragmentation, and extended transcript hash are detailed in the subsection below and in Section 7.

Remark 2. *The implementation hashes the full TLV-encoded AUTH0 command and response (including **trans-id**, **reader-id**, and UD noise) using SHAKE256-512 [24] rather than the simplified $H(pk_e || c_e)$ in the formal model. The additional*

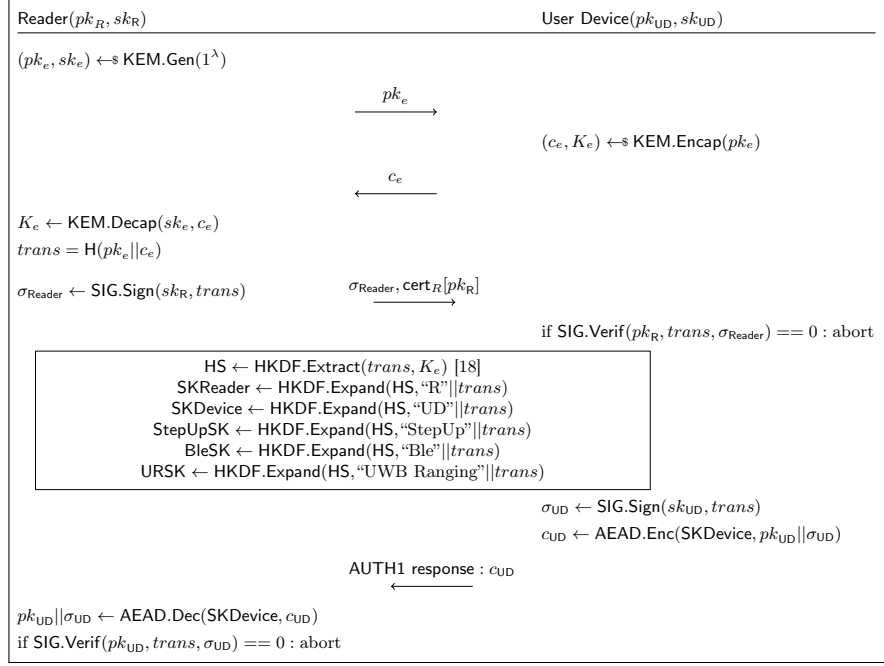


Fig. 2. Our PQ-Aliro authenticated key exchange protocol. The KEM provides ephemeral key establishment; the signature scheme provides mutual authentication. Both parties derive five session keys (SK_{Reader} , SK_{Device} , $StepUpSK$, $BleSK$, $URSK$) from the shared handshake secret HS via HKDF [18].

fields strengthen session binding without affecting the security argument. Specifically, modelling H as a collision-resistant hash function (Definition 1), including these fields in the hash input preserves session freshness and transcript uniqueness; since the collision-resistance argument in Game 1 of Section 6 applies to any extension of the transcript, the security reduction carries over unchanged.

4.1 APDU Message Mapping

The four logical messages of the PQ-Aliro Expedited Phase do not map directly to single ISO-DEP APDUs. Because post-quantum cryptographic objects far exceed the 255-byte ISO/IEC 14443 APDU payload limit, every message must be fragmented into a chain of command or response APDUs before transmission. On the Reader (command) side the implementation sets $\text{MAX-CHUNK-SIZE} = 200\text{ B}$ for AUTH0 ($\text{INS} = 0x80$) and AUTH1 ($\text{INS} = 0x81$), and $\text{MAX-CHUNK-SIZE} = 255\text{ B}$ for LOADCERT ($\text{INS} = 0xD1$). Each intermediate chunk carries $P1 = 0x10$ to signal that further data follows; the final chunk sets $P1 = 0x00$. On the User Device (response) side, $\text{MAX-APDU-CHUNK} = 240\text{ B}$; the UD appends status

Table 2. PQ-Aliro APDU message mapping (ML-DSA-65 $\times$ ML-KEM-768, default configuration)

Message	Dir.	INS	Payload Summary	Chunk
AUTH0 Cmd	R→UD	0x80	pk_e (1 184 B), $trans-id$ (16 B), $reader-id$ (32 B)	200 B
AUTH0 Resp	UD→R	—	KEM ciphertext (1 088 B), UD noise (32 B)	240 B
LOADCERT Cmd	R→UD	0xD1	Profile0000 cert (approximately 5 500 B): pk_R (1 952 B), σ (3 309 B), DN (approximately 200 B)	255 B
LOADCERT Resp	UD→R	—	0x9000 only	—
AUTH1 Cmd	R→UD	0x81	σ_{reader} (3 309 B) over transcript	200 B
AUTH1 Resp	UD→R	—	AEAD($pk_{ud} \parallel \sigma_{ud} \parallel bmap$), + 16 B tag	240 B

word 0x61 $\mathbf{xx}$ to every non-final response chunk and 0x9000 to the last. The Reader retrieves subsequent chunks with a GET RESPONSE (INS = 0xC0) command.

Fields whose encoded length exceeds 255 bytes are wrapped in BER-TLV long-form length encoding: a leading 0x82 byte followed by a two-byte big-endian length, adding 4 bytes of framing overhead per such field. This affects the ML-KEM-768 ephemeral public key (1 184 B $\rightarrow$ 1 188 B on the wire), the KEM ciphertext (1 088 B $\rightarrow$ 1 092 B), and all ML-DSA-65 signatures (3 309 B $\rightarrow$ 3 313 B).

Table 2 enumerates the six APDU-layer messages of a single PQ-Aliro transaction under the default ML-DSA-65 $\times$ ML-KEM-768 configuration.

Under this configuration the six messages collectively require approximately 75 APDUs and transfer approximately 16.4 KB of application-layer data, representing a $\sim 14\times$ increase over the approximately 1.2 KB of the classical ECC instantiation. The dominant contributors are the LOADCERT round (approximately 5 500 B, 22 APDUs) and the AUTH1 Response (approximately 5 289 B, 23 APDUs), both driven by the large size of ML-DSA-65 public keys and signatures.

Relation to the security model. Figure 2 shows that both parties derive five session keys from the handshake secret HS: SKDevice (NFC record-layer encryption), SKReader, StepUpSK, BleSK, and URSK (BLE and UWB channels). The formal security model in Section 5 abstracts this key schedule into a single session key k_i^s , which corresponds to SKDevice—the key used for the AUTH1 Response AEAD encryption and therefore the key whose indistinguishability is the critical security goal. The remaining four keys (SKReader, StepUpSK, BleSK, URSK) are derived via independent HKDF.Expand calls from the same HS; their security follows from the PRF security of HKDF.Expand (Definition 5) under the standard key-separation argument, and does not require a multi-stage AKE model.

5 Security Model

The expedited-standard phase is designed to achieve the following security objectives:

- Mutual authentication.
- Forward secrecy (under long-term key corruption).
- Tracking resilience.
- Integrity and confidentiality.

Tracking resilience. To realise tracking resilience, any information that could serve as a stable identifier for a User Device (e.g., its identity) must be confidentiality-protected while in transit. The User Device SHALL construct an AUTH1 response that prevents an external observer, by inspecting the response, from inferring whether the device knows a particular reader public key or Access Credential. Concretely, the AUTH1 response is encrypted, thereby thwarting a malicious Reader from extracting persistent identifiers or testing whether a prior relationship with a specific Reader exists.

Our security model builds upon the framework of Schäge et al. [30], which extends the Bellare–Rogaway (BR) model [2] to naturally capture long-term key compromise and mutual authentication. Since Aliro does not aim to resist compromise of *session state* or *ephemeral randomness*, we do not grant the adversary these corruption capabilities. To incorporate tracking resilience, the model explicitly accounts for the privacy of User Device identities. By contrast, the Reader’s identity need not remain hidden since the Reader certificate is transmitted in the clear.

In the security mode of [30], there are two ways to select which identity should be used. However, in our case, there is only one way, i.e., each party decides the identity on its own. Moreover, the public key of the party instead of the to-be-used identity needs to be transferred to the partner. Each party has its unique role, i.e., the initiator or responder.

We extend the model by introducing a criterion for indistinguishability of the responder identity used in the protocol. To achieve this, we equip each responder party with two different identities. Then we define a local variable, i.e., the selector bit d that models which identity is used by the responder to authenticate data with. However, different from the model in [30], we assume that the identity used by the responder is always decided on its own.

To model the privacy of the responder identity, the adversary may request two different challenges from the challenger with an extended **Test**-query:

- By asking $\text{Test}(\pi_i^s, \text{KEY})$, he requests the classical key indistinguishability;
- By asking $\text{Test}(\pi_i^s, \text{ID})$, he requests an identity indistinguishability challenge.

We follow Schäge et al. to define a query **Unmask** that allows the adversary to learn the identity of the responder in an oracle.

Computational model for AKE. The execution environment is parameterised by a security parameter λ ; all parties as well as the adversary are modelled as PPT algorithms.

Execution environment. Let $\mathcal{P} = \{P_1, \dots, P_n\}$ be the set of protocol participants. Each initiator party P_i possesses an identity ID_i and a long-lived key pair (sk_i, pk_i) . Each responder party P_i possesses two identities ID_i^0 and ID_i^1 , and each of these identities is associated with a long-lived key pair (sk_i^b, pk_i^b) for $b \in \{0, 1\}$. During execution, P_i may spawn several process instances, called *oracles*, denoted $\{\pi_i^s : s \in \{1, \dots, \ell\}\}$. The subscript and superscript indicate that π_i^s is the s -th oracle of party P_i ; P_i is also referred to as the *holder* of π_i^s . An oracle that sends the first protocol message is called the *initiator*; otherwise it is the *responder*.

Local and global variables. Each oracle π_i^s shares the global variables of its holder P_i and may use P_i 's secret keys for decryption or signing. It maintains the following local variables in its own memory:

- A session key $k = k_i^s$.
- The role of the holder P_i : $\text{role} = \text{init}/\text{resp}$.
- The identity selector bit $d = d_i^s \in \{0, 1\}$, which determines that identity ID_i^d (and sk_i^d) is used in the protocol run.
- An intended partner indicator $\text{Partner} = (j, f)$, holding a pointer $j \in [1; n]$ to the party $P_j \in \{P_1, \dots, P_n\}$ (often called the *peer*) and a partner identity bit $f \in \{0, 1\}$. The value Partner_i^s indicates that the public key pk_j^f is used by π_i^s to authenticate the received messages. If P_i is a responder, we always set $f = 0$.

All local variables are initialized to a distinguished symbol $\perp$, which lies outside any valid domain. During a protocol run, each oracle updates its variables according to the specification and finally either *accepts* or *rejects*. The ultimate purpose of an AKE protocol is to establish a session key k . The adversary $\mathcal{A} \notin \{P_1, \dots, P_n\}$ is a special entity that can interact with all oracles by issuing queries.

Adversarial capabilities. The capabilities granted to the adversary are defined as follows.

- $\text{Send}(\pi_i^s, m)$: The active adversary can deliver an arbitrary message m to oracle π_i^s , which responds according to the protocol. If $m = (\emptyset, P_j)$, where $\emptyset$ is a distinguished start string, π_i^s is activated as initiator with intended partner P_j and returns the first protocol message.
- $\text{Reveal}(\pi_i^s)$: It allows the adversary to obtain the session key k_i^s computed by π_i^s . This query is only answered if $k_i^s \neq \perp$, i.e., if the oracle has accepted.
- $\text{Unmask}(\pi_i^s)$: If P_i is an initiator, the adversary is given the identity selector bit of P_i , otherwise, the adversary is given the identity selector bit of P_i 's peer in π_i^s .
- $\text{Corrupt}(P_i)$: The adversary may query any oracle of P_i and receive the long-term key pair (sk_i, pk_i) . Afterwards, P_i is marked *corrupted*.
- $\text{Test}(\pi_i^s, m)$: This query may be issued at most once; the oracle is then called the *Test-oracle*. If π_i^s has not yet accepted, $\perp$ is returned. Otherwise a fair coin b is flipped and this oracle proceeds as follows.

- $m = \text{KEY}$: If $b = 0$ the real session key k_i^s is returned; if $b = 1$ a uniformly random element from the keyspace is returned. The result is called the *candidate key*.
- $m = \text{ID}$: The adversary is given $d_i^s \oplus b$. In this case, the output of the Test oracle is called *candidate identity selector bit*. If $m = \text{ID}$, it is required that P_i is a responder. If P_i is an initiator, the query is invalid and the Test oracle outputs $\perp$.

Original key partnering. To exclude trivial attacks, various partnering definitions have been proposed in the literature, starting from matching conversations [2] and session identifiers [4], up to the distinction between contributive and matching identifiers [10]. Recently, Li and Schäge [22] demonstrated that matching-conversation-based notions are susceptible to so-called no-match attacks. We therefore adopt their conceptually novel partnering definition, called *original key partnering*, which is independent of the concrete message flow of the protocol. As argued in [22], this approach leads to conceptually simpler security proofs because only a subset of active attacks needs to be analyzed.

Definition 7 (Original Key). *For a pair of communicating oracles π_i^s (initiator) and π_j^t (responder), the original key is the session key that would be computed by each oracle in a protocol run with a completely passive adversary. We write $\text{Origin}(\pi_i^s, \pi_j^t)$ for this key.*

Definition 8 (Original Key Partnering). *Two oracles π_i^s and π_j^t are said to be partnered if both have computed their original key $\text{Origin}(\pi_i^s, \pi_j^t)$.*

Let $\mathcal{M}_i^s$ denote the set of all oracles partnered with π_i^s .

Security game. We define the security of an AKE protocol via a game. The protocol is considered insecure if an efficient (PPT) adversary can win the game with non-negligible probability.

Definition 9 (Security Game). *The following game is played between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.*

1. $\mathcal{C}$ simulates n parties P_i , $i \in \{1, \dots, n\}$. For each initiator party P_i , $\mathcal{C}$ generates an identity ID_i and a long-lived key pair $(\text{sk}_i, \text{pk}_i)$. For each responder party P_i , $\mathcal{C}$ generates two identities ID_i^0 and ID_i^1 , and each of these identities is associated with a long-lived key pair $(\text{sk}_i^b, \text{pk}_i^b)$ for $b \in \{0, 1\}$.
2. $\mathcal{A}$ may adaptively issue *Send*, *Reveal*, *Unmask*, *Corrupt*, and a single *Test* query to any oracle π_i^s ($i \in [n]$, $s \in [\ell]$).
3. Finally, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ for the hidden bit b .

Definition 10 (Secure AKE). *Let $\mathcal{A}$ be a PPT adversary interacting with $\mathcal{C}$ in the security game above. Suppose $\mathcal{A}$ calls $\text{Test}(\pi_i^s)$, which internally uses bit b , and let $\text{Partner}_i^s = P_j$ be the intended partner of π_i^s . Let b' be $\mathcal{A}$'s output. We say that $\mathcal{A}$ wins the game if $b = b'$ and one of the following holds:*

- $m = \text{KEY}$, then the following conditions are all satisfied:

1. $\text{Reveal}(\pi_i^s)$ has never been queried;
 2. no $\text{Reveal}(\pi_j^t)$ query has been asked for any oracle π_j^t partnered with π_i^s ;
 3. $\text{Partner}_i^s = P_j$ has not been corrupted while $\mathcal{M}_i^s = \emptyset$ (i.e., there is no partnered oracle).
- $m = \text{ID}$, then we require that
- no query $\text{Unmask}(\pi_i^s)$ has been asked;
 - $\text{Partner}_i^s = (P_j, f)$ has not been corrupted while $M_i^s = \emptyset$.

Define $\text{Adv}_\Pi^{\text{AKE-sec}}(\mathcal{A}, \lambda) = |\Pr[b = b'] - 1/2|$. An AKE protocol Π is secure if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_\Pi^{\text{AKE-sec}}(\mathcal{A}, \lambda) \leq \text{negl}(\lambda).$$

Observe that our model provides all attack queries present in the original Bellare–Rogaway model [2]. Moreover, it captures important variants involving secret key corruption. To model key compromise impersonation (KCI) attacks [19], we permit the adversary to corrupt the holder of the Test-oracle at any time. Allowing corruption of the long-term keys of oracles (subject to the usual trivial-attack restrictions) models (full) perfect forward secrecy. Finally, because no restriction is placed on the relationship between the initiator’s and responder’s key material, our model also considers reflection attacks [19], where a party communicates with itself (e.g., two devices sharing the same long-term secret).

6 Security Proof

We prove that PQ-Aliro is secure in the above security model.

Theorem 1. *Let n be the number of parties and t be the number of sessions per party. Assume the signature is EUF-CMA-secure, the HKDF is PRF-secure, the KEM is IND-1CCA secure. Then, for any PPT adversary $\mathcal{A}$ against the PQ-Aliro protocol, we have*

$$\text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-sec}}(\mathcal{A}, \lambda) \leq 2 \cdot \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda) + 2n^2t^2 \cdot (\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda)) + 2n^2t \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda) + \text{Adv}_{\text{AEAD}}^{\text{AEAD-sec}}(\mathcal{B}_5, \lambda)$$

where $\mathcal{B}_1$ – $\mathcal{B}_5$ are adversaries against, respectively, collision resistance of H , EUF-CMA of SIG , IND-1CCA of KEM , PRF of HKDF , and AEAD security.

Proof sketch. We distinguish between two adversary types:

- The initiator adversary $\mathcal{A}_{\text{init}}$, whose goal is to correctly guess the outcome of $\text{Test}(\pi_i^s, m)$ when π_i^s plays the initiator role;
- The responder adversary $\mathcal{A}_{\text{resp}}$, which wins by predicting the result of $\text{Test}(\pi_i^s, m)$ for a responder session π_i^s .

We prove that all derived keys are pseudorandom and the privacy preserving property of the user device identity. The argument builds upon the anonymous key-exchange paradigm enhanced with digital signatures for identity authentication.

Proof. Let $\mathcal{A}_{\text{init}}$ be any probabilistic polynomial-time initiator adversary attacking the PQ-Aliro protocol depicted in Figure 2. Then it is obvious that

$$\text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-sec}}(\mathcal{A}_{\text{init}}, \lambda) \leq \text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-key}}(\mathcal{A}_{\text{init}}, \lambda) + \text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-pp}}(\mathcal{A}_{\text{init}}, \lambda),$$

where $\text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-key}}(\mathcal{A}_{\text{init}}, \lambda)$ and $\text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-pp}}(\mathcal{A}_{\text{init}}, \lambda)$ are the advantage of $\mathcal{A}$ in winning the AKE security game defined in Definition 9 by correctly answering $\text{Test}(\cdot, \text{KEY})$ and $\text{Test}(\cdot, \text{ID})$, respectively.

Next we need to prove Lemma 1 and Lemma 2.

Lemma 1. *Let $\mathcal{A}_{\text{init}}$ be any probabilistic polynomial-time adversary attacking the PQ-Aliro protocol depicted in Figure 2. Then the probability that $\mathcal{A}_{\text{init}}$ answers $\text{Test}(\cdot, \text{KEY})$ -challenge correctly is no more than $\frac{1}{2} + \text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-key}}(\mathcal{A}_{\text{init}}, \lambda)$, where*

$$\text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-key}}(\mathcal{A}_{\text{init}}, \lambda) \leq 2 \cdot \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda) + 2n^2t^2 \cdot (\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda)) + 2n^2t \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda)$$

where $\mathcal{B}_1$ – $\mathcal{B}_4$ are adversaries against collision resistance of H , EUF-CMA of SIG , IND-1CCA of KEM , and PRF of HKDF .

We define a sequence of games between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$, where the challenger runs the PQ-Aliro protocol in the first game, bounding the differences between games via a series of assumptions. In the end, we demonstrate that $\mathcal{A}$'s advantage in winning the final game is 0. Let Adv_i denote the advantage of $\mathcal{A}$ in the Game i , where $i = 1, 2, \dots$.

Game 0. This game is exactly the AKE security experiment of Definition 9. Consequently,

$$\text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-key}}(\mathcal{A}_{\text{init}}, \lambda) = \text{Adv}_0. \quad (1)$$

Game 1. The challenger halts if two honest sessions produced an identical hash value from distinct inputs. If such a collision appear, we immediately obtain an adversary $\mathcal{B}_1$ breaking the collision resistance of H by outputting the two colliding preimages. Hence,

$$|\text{Adv}_0 - \text{Adv}_1| \leq \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda). \quad (2)$$

Game 2. We now guess the initiator session and its intended peer that will be tested. The challenger samples $(i^*, s^*, j^*) \leftarrow_{\$} [n] \times [t] \times [n]$ uniformly at random. It aborts if the adversary's Test query does not match $(\pi_{i^*}^{s^*}, j^*)$, i.e., if $(i, s) \neq (i^*, s^*)$, or if $\pi_{i^*}^{s^*}$ is not an initiator, or if its peer is not j^* . We have

$$\text{Adv}_1 \leq n^2t \cdot \text{Adv}_2. \quad (3)$$

Game 3. We define the event **Forge**: the initiator session $\pi_{i^*}^{s^*}$ accepts a valid signature σ^* from party j^* , yet no responder oracle of j^* ever generated that signature. In this game the challenger aborts upon **Forge**. **Games 2** and **Game 3** are identical until **Forge** occurs, so

$$|\text{Adv}_2 - \text{Adv}_3| \leq \Pr[\text{Forge}]. \quad (4)$$

We bound $\Pr[\text{Forge}]$ by constructing an EUF-CMA forger $\mathcal{B}_2$ against the signature scheme. $\mathcal{B}_2$ receives a challenge public key pk^* and sets $\text{pk}_{j^*} \leftarrow \text{pk}^*$ for the intended peer. All other key pairs are generated honestly. $\mathcal{B}_2$ runs the AKE environment for $\mathcal{A}$, forwarding signing requests for pk_{j^*} to its own signing oracle. If $\mathcal{A}$ ever produces a valid signature under pk_{j^*} on a fresh message (the transcript prevents replays because hash collisions are already excluded), $\mathcal{B}_2$ submits it as a forgery. Therefore,

$$\Pr[\text{Forge}] \leq \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda). \quad (5)$$

Combining (4) and (5) yields

$$|\text{Adv}_2 - \text{Adv}_3| \leq \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda). \quad (6)$$

From now on we may assume that all signatures received by the initiator were actually generated by the intended responder session.

Game 4. We further guess the specific responder oracle $\pi_{j^*}^{t^*}$ of P_{j^*} that issued the signature accepted by $\pi_{i^*}^{s^*}$. Since at most t sessions exist per party, we obtain

$$\text{Adv}_3 \leq t \cdot \text{Adv}_4. \quad (7)$$

Game 5. In this game, we replace the ephemeral shared secret K_e in session $\pi_{j^*}^{t^*}$ by a uniformly random $\tilde{K}_e$. If $\pi_{i^*}^{s^*}$ receives the same ciphertext c_e that $\pi_{j^*}^{t^*}$ sent, its K_e is also set to $\tilde{K}_e$. If the adversary $\mathcal{A}$ can detect this change with non-negligible probability, then we can build an adversary $\mathcal{B}_3$ that breaks the IND-1CCA security of KEM with non-negligible probability as follows.

$\mathcal{B}_3$ obtains a challenge (pk^*, c^*, K^*) from its challenger. It embeds pk^* as pk_e of the target sessions, sets $c_e \leftarrow c^*$ in $\pi_{j^*}^{t^*}$, and uses K^* as the derived K_e . If $\mathcal{A}$ delivers c^* faithfully, $\mathcal{B}_3$ assigns the same K^* to $\pi_{i^*}^{s^*}$; otherwise it queries its decapsulation oracle on the modified ciphertext. When K^* is the real key the view of the adversary matches **Game 4**, and when K^* is random the view matches **Game 5**. Thus,

$$|\text{Adv}_4 - \text{Adv}_5| \leq \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda). \quad (8)$$

Game 6. Next, the final session key $K = \text{HKDF.Expand}(\text{HS}, \text{trans})$ is replaced by a random K^* . When both $\pi_{i^*}^{s^*}$ and $\pi_{j^*}^{t^*}$ compute the same handshake secret HS, we also set the responder's output to K^* . A distinguisher between **Game 5** and **Game 6** yields an adversary $\mathcal{B}_4$ against the pseudorandomness of HKDF. Therefore, we have

$$|\text{Adv}_5 - \text{Adv}_6| \leq \text{Adv}_{\text{HKDF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda). \quad (9)$$

Moreover, in **Game 6**, the Test query always returns a uniformly random key, so we have

$$\text{Adv}_6 = 0. \quad (10)$$

Combining the above security bounds, we have:

$$\text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-key}}(\mathcal{A}_{\text{init}}, \lambda) \leq \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda) + n^2 t \cdot \left[t \cdot (\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda)) + \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda) \right]$$

Next, we prove the tracking resilience property of PQ-Aliro.

Lemma 2. *For any PPT adversary $\mathcal{A}_{\text{init}}$ against the PQ-Aliro protocol in Figure 2, the probability that $\mathcal{A}_{\text{init}}$ answers the $\text{Test}(\cdot, \text{ID})$ -challenge of some oracle correctly is at most $\frac{1}{2} + \text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-pp}}(\mathcal{A}_{\text{init}}, \lambda)$ with*

$$\begin{aligned} \text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-pp}}(\mathcal{A}_{\text{init}}, \lambda) \leq & \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda) + n^2 t^2 \cdot (\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda)) \\ & + n^2 t \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda) + \text{Adv}_{\text{AEAD}}^{\text{AEAD-sec}}(\mathcal{B}_5, \lambda), \end{aligned}$$

where $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{B}_4$ and $\mathcal{B}_5$ denote, respectively, adversaries against the collision resistance of H , the EUF-CMA security of SIG , the IND-1CCA security of KEM , the PRF security of HKDF and the security of AEAD.

Proof. The privacy of the responder identity corresponds to the key used for AEAD. Therefore, we start-off with the security game after **Game 6** in the proof of Lemma 3.

Game 0. In this game, we extend **Game 6** in the proof of Lemma 3. We have

$$\begin{aligned} \text{Adv}_0 \leq & \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda) + n^2 t^2 \cdot (\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda)) \\ & + n^2 t \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda). \end{aligned}$$

After **Game 0**, the encryption key for AEAD is chosen at uniformly random by the Test -oracle $\pi_{i^*}^{s^*}$.

Game 1. In this game, we replace $d = d_{j^*}^{t^*}$ with $d' = d_{j^*}^{t^*} \oplus 1$, and replace the signature generated by the responder that was constructed using $sk_{i^*}^{(d)}$ by a signature that uses the other secret key $sk_{i^*}^{(d')}$. If the adversary can distinguish **Game 1** from **Game 0**, then we can construct another adversary $\mathcal{B}_5$ that breaks the security of AEAD. $\mathcal{B}_5$ sets $m_0 \leftarrow (pk_{\text{UD}} || \sigma_{\text{UD}})$ which corresponds to ID_d , and $m_1 \leftarrow (pk_{\text{UD}} || \sigma_{\text{UD}})$ which corresponds to $\text{ID}_{d'}$. Then $\mathcal{B}_5$ sends m_0, m_1 to its challenger in the security experiment of AEAD. Upon receiving the ciphertext c^* from the challenger, $\mathcal{B}_5$ sets c^* as the AUTH1 response message of $\pi_{j^*}^{t^*}$. $\mathcal{B}_5$ generates $b \leftarrow \{0, 1\}$, computes $w \leftarrow d_{j^*}^{t^*} \oplus b$ and set w as the response to $\text{Test}(\pi_{i^*}^s, \text{ID})$. After receiving b' from $\mathcal{A}$, $\mathcal{B}_5$ returns $w \oplus b'$ to its challenger. We have

$$\text{Adv}_0 \leq \text{Adv}_1 + \text{Adv}_{\text{AEAD}}^{\text{AEAD-sec}}(\mathcal{B}_5, \lambda).$$

In **Game 1**, since the responder identity has been replaced, the adversary has no advantage in winning the security game. Therefore, we have

$$\text{Adv}_1 = 0.$$

In conclusion, we have

$$\begin{aligned} \text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-pp}}(\mathcal{A}, \lambda) &\leq \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda) + n^2 t^2 \cdot (\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda)) \\ &\quad + n^2 t \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda) + \text{Adv}_{\text{AEAD}}^{\text{AEAD-sec}}(\mathcal{B}_5, \lambda). \end{aligned}$$

□

For Responder-adversary, we have the following lemma.

Lemma 3. *Let $\mathcal{A}_{\text{resp}}$ be any probabilistic polynomial-time responder adversary attacking the PQ-Aliro protocol depicted in Figure 2. Then*

$$\begin{aligned} \text{Adv}_{\text{PQ-Aliro}}^{\text{AKE-sec}}(\mathcal{A}_{\text{resp}}, \lambda) &\leq 2 \cdot \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1, \lambda) + 2n^2 t^2 \cdot (\text{Adv}_{\text{PRF}}^{\text{PRF-sec}}(\mathcal{B}_4, \lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_3, \lambda)) \\ &\quad + 2n^2 t \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_2, \lambda) + \text{Adv}_{\text{AEAD}}^{\text{AEAD-sec}}(\mathcal{B}_5, \lambda), \end{aligned}$$

where $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{B}_4$ and $\mathcal{B}_5$ denote, respectively, adversaries against the collision resistance of H , the EUF-CMA security of SIG , the IND-1CCA security of KEM , the PRF security of HKDF and the security of AEAD .

The proof proceeds with the same game-hopping strategy, swapping the roles of initiator and responder in the reduction arguments. We omit the repetitive details.

□

7 Implementation

This section describes the engineering realisation of PQ-Aliro across two platforms: an Android Host Card Emulation (HCE) application acting as the User Device (UD) and a PC-side NFC reader application. Both implementations are open source and written in Java.

Engineering challenges. Porting PQ-Aliro to Android HCE introduces three challenges absent at the protocol level: (i) Key isolation: Dilithium private keys cannot be stored in Android Keystore natively and must be protected against JVM garbage-collection non-determinism that can expose heap contents; (ii) Deadlock prevention: the HCE main thread cannot block on ML-DSA signing without triggering an Application Not Responding (ANR) event; (iii) APDU fragmentation: post-quantum objects (up to 3,309 B for an ML-DSA-65 signature) must be split across chains of 200–255 B APDUs with correct continuation signalling. The following subsections describe the solutions adopted for each challenge.

7.1 System Architecture

The UD application runs on Android and exposes the PQ-Aliro applet via the Host Card Emulation framework (`HostApuService`). The Reader application

runs on a standard PC equipped with an NFC reader peripheral and communicates with the UD over the ISO/IEC 14443 Type-A contactless channel. Both sides share a common message state machine that advances through four stages: $\text{SELECT} \rightarrow \text{AUTH0} \rightarrow \text{LOADCERT} \rightarrow \text{AUTH1}$. Each stage corresponds to a pair of command and response APDUs as detailed in Section 4.1.

All post-quantum cryptographic operations are provided by BouncyCastle 1.78 with the BCPQC extension. On the UD, the provider is registered at application start-up:

```
Security.addProvider(new BouncyCastlePQCProvider());
```

The same provider is used on the PC Reader. No native (JNI) code is required: both ML-KEM and ML-DSA are implemented entirely in pure Java within the BouncyCastle library, making the codebase portable across any JVM-capable platform. The UD manages long-term Dilithium key material independently of Android Keystore, since the Android Keystore API does not yet expose lattice-based algorithm support; all PQC key operations therefore remain in the application layer, with the Keystore used only to protect symmetric encryption keys (see Section 7.3).

7.2 ML-KEM Integration

ML-KEM [25] is realised using the CRYSTALS-Kyber [3] implementation in BouncyCastle BCPQC. The Reader generates a fresh ephemeral key pair for every transaction:

```
KeyPairGenerator kpg =
    KeyPairGenerator.getInstance("Kyber", "BCPQC");
kpg.initialize(KyberParameterSpec.kyber768, new SecureRandom());
KeyPair ephemeral = kpg.generateKeyPair();
```

The 1184-byte public key is extracted as a raw byte array (the SPKI outer wrapper is stripped) and transmitted in the AUTH0 Command under TLV tag 0x87. On the UD, the received bytes are reconstructed into a `KyberPublicKeyParameters` object via a helper `decodeKyberPublicKey(rawBytes)`, which re-attaches the algorithm identifier before passing the key to the JCA layer.

Encapsulation on the UD uses a `KeyGenerator` with a `KEMGenerateSpec`:

```
KeyGenerator kg =
    KeyGenerator.getInstance("Kyber", "BCPQC");
kg.init(new KEMGenerateSpec(readerPublicKey, "AES"));
SecretKeyWithEncapsulation ske =
    (SecretKeyWithEncapsulation) kg.generateKey();
```

The 1088-byte encapsulation c is returned in the AUTH0 Response under tag 0x86. The 32-byte shared secret S_e is immediately imported into the Android Keystore under alias `SSKey`; the raw byte array is then zeroed with `Arrays`.

`fill(rawBytes, (byte) 0x00)` to prevent the secret from persisting on the JVM heap. The Reader performs decapsulation symmetrically via `KEMExtractSpec` to recover S_e , after which both parties derive the session key $SK - device$ through the KDF chain described in Section 4.

7.3 ML-DSA Integration and Key Isolation

ML-DSA [26] is realised from the CRYSTALS-Dilithium submission [8] in BouncyCastle BCPQC.

Reader-side signing. On the PC Reader, the long-term Dilithium private key is held in process memory throughout the application lifetime. Signing uses the standard JCA `Signature` interface:

```
Signature signer =
    Signature.getInstance("Dilithium", "BCPQC");
signer.initSign(readerPrivateKey);
signer.update(transcriptBytes);
byte[] sigma = signer.sign();
```

UD-side key isolation. The UD cannot safely hold the Dilithium private key in the main application process. The root cause is that `DilithiumPrivateKeyParameters` does not implement `javax.security.auth.Destroyable` in BouncyCastle 1.78; calling `destroy()` leaves the key material on the JVM heap until the garbage collector decides to overwrite the memory region, with no timing guarantee. A compromised main process could therefore read the private key from heap memory at any point between key generation and GC collection.

To eliminate this attack surface, the UD delegates all Dilithium signing to a dedicated `PQCSignatureService` that runs in a separate named process declared with the Android manifest attribute `android:process=":pqc-signer"`. A named process runs in a separate Linux process with its own address space and shares the application's UID, but physically isolates all private-key operations from the main process's memory. This is distinct from `android:isolatedProcess="true"`, which would assign a distinct UID and deny file-system access; the named-process approach is used here because `PQCSignatureService` requires access to `SharedPreferences` for encrypted key retrieval. The main process sends the message bytes to be signed over a Binder IPC interface; `PQCSignatureService` loads the encrypted private key (see below), performs the signing operation entirely within its own address space, returns the 3309-byte signature over the same Binder channel, and immediately calls `Process.killProcess(Process.myPid())` to terminate itself, limiting the key-exposure window to the duration of a single signing invocation (typically ≤ 150 ms on mid-range Android hardware).

Because the entire lifecycle of the named process—from private key loading to process exit—is bounded to a single signing call, the Dilithium private key material never appears in the main application's address space. A crash

or compromise of the main process cannot access the signing key held in the `:pqc-signer` address space.

Persistent key storage. The Dilithium private key cannot be stored in Android Keystore directly. Instead, it is stored in `SharedPreferences` encrypted under AES-256-GCM [9]: the raw private key bytes are encrypted once at provisioning time (KEY-KYB-PRIV-CT stores the ciphertext, KEY-KYB-PRIV-IV stores the GCM IV), and the 256-bit AES wrapping key is held in Android Keystore, which provides hardware-backed protection on devices with a Trusted Execution Environment. The isolated process retrieves and decrypts the private key on each invocation, uses it exactly once, then discards it along with the process.

7.4 NFC Fragmentation and Asynchronous Signing

APDU fragmentation. As described in Section 4.1, PQ-Aliro payloads must be split into chains of APDUs to respect the ISO/IEC 14443 ISO-DEP frame limit. The Reader sends AUTH0 and AUTH1 command chunks of at most 200 bytes, and LOADCERT chunks of at most 255 bytes. The UD sends response chunks of at most 240 bytes. Fields longer than 255 bytes are encoded with the BER-TLV long-form length prefix `0x82 len-hi len-lo`, adding 4 bytes of overhead per affected field. This encoding is applied to the ML-KEM-768 ephemeral public key (tag `0x87`), the KEM ciphertext (tag `0x86`), all ML-DSA-65 public keys (tags `0x5A`), and all ML-DSA-65 signatures (tag `0x9E`) in both AUTH1 Command and AUTH1 Response.

Asynchronous signing to prevent HCE deadlock. Android’s HCE framework invokes `processCommandAdu()` on a dedicated system thread. If this callback blocks on a synchronous Binder call to an isolated process, the system-thread call stack is held indefinitely: the Reader’s NFC controller times out the APDU exchange, and Android may trigger an Application Not Responding (ANR) dialog. To avoid this, the UD adopts a non-blocking response strategy for the AUTH1 Command. Upon receiving the final chunk of AUTH1, `processCommandAdu()` spawns a background thread to perform the Binder-based signing call and immediately returns the two-byte response `0x61 0x00` (“more data available, zero bytes ready”) to the NFC system thread, releasing it at once. The Reader interprets `0x61 xx` as a continuation indicator and enters a polling loop, sending `GET RESPONSE (INS = 0xC0)` commands at regular intervals. Once the background signing thread completes and writes the fully assembled AUTH1 Response into the `responseQueue`, the next `GET RESPONSE` retrieves the first 240-byte chunk, with subsequent `GET RESPONSE` commands collecting the remaining 22 chunks.

AEAD and IV management. The AUTH1 Response payload is protected by AES-256-GCM using the session key $SK - device$. The GCM IV is derived as the 12-byte value `0x00 00 00 00 00 00 00 01 || ctr`, where `ctr` is a 4-byte big-endian counter initialised to 1 and incremented for each subsequent AEAD invocation

within a session. The IV is not transmitted over NFC; both parties derive it independently from the shared counter state. The 16-byte GCM authentication tag is appended to the ciphertext, yielding a 5 289-byte AUTH1 Response payload. All citation to the Aliro 1.0 specification [6] for the APDU command structure and status-word conventions has been preserved throughout this implementation.

7.5 End-to-End Testing

The implementation is validated by a comprehensive end-to-end test suite that exercises all nine PQC parameter combinations ($\text{Dilithium}\{2, 3, 5\} \times \text{Kyber}\{512, 768, 1024\}$).

The test harness runs entirely in-memory: the PC-side Reader and the Android HCE logic are instantiated within the same JVM process, with a byte-array channel substituting for the physical NFC interface. This removes the need for real NFC hardware and makes the tests fully reproducible in a standard CI environment.

Each test configuration exercises 20 assertions covering: (i) KEM correctness—that the shared secret S_e recovered by the Reader via **KEM ExtractSpec** matches the value encapsulated by the UD; (ii) transcript hash consistency—that $\text{transAH0} = \text{SHAKE256-512}(\text{AUTH0-cmd} \parallel \text{AUTH0-resp})$ [24] is identical on both sides; (iii) KDF consistency—that $SK - \text{device}$ derived by both parties is identical; (iv) AEAD round-trip correctness; (v) ML-DSA signature verification in both directions (Reader signature verified by UD, UD signature verified by Reader); and (vi) serialisation integrity of all TLV fields across the fragmentation and reassembly pipeline.

All 20 assertions pass across all nine parameter combinations, confirming the correctness of the implementation for the full range of NIST security levels supported by PQ-Aliro [34]. In addition to the positive assertions, the harness includes negative test cases that verify protocol abort upon: (a) a tampered transcript hash in the AUTH1 Command, (b) an invalid ML-KEM ciphertext in the AUTH0 Response, and (c) a forged ML-DSA-65 signature on the Reader certificate.

These implementation results provide the foundation for the quantitative communication-overhead analysis and NFC latency estimates presented in the following section.

8 Evaluation

We evaluate PQ-Aliro along three complementary axes: the raw communication overhead introduced by post-quantum cryptographic primitives, the resulting NFC transmission latency, and the security-level vs. overhead trade-off across all nine parameter combinations. All byte counts are derived from static analysis of the implemented APDU message structures; no physical testbed measurement is required for the core overhead figures, though we identify empirical validation as future work.

Table 3. Per-phase communication overhead for the default PQ-Aliro configuration (ML-DSA-65 $\times$ ML-KEM-768, NIST Level 3). Chunk sizes: Reader ≤ 200 B/APDU, UD ≤ 240 B/APDU.

Message	APDUs	App. bytes	Link bytes
AUTH0 Command	7	1,250	1,292
AUTH0 Response	5	1,126	1,136
LOADCERT Command approximately	22 approximately	5,500 approximately	5,632 approximately
LOADCERT Response	1	2	4
AUTH1 Command	17	3,316	3,418
AUTH1 Response	23	5,289	5,335
Total	≈ 75	$\approx 16,483 \approx 16,817 \approx 16.4$ KB	

8.1 Communication Overhead Analysis

Methodology. We adopt the following counting conventions throughout this section. (1) Application-layer bytes denote the TLV payload body, excluding APDU command headers and status words. (2) Link bytes add the per-frame framing cost: 6 B per command APDU (CLA $\parallel$ INS $\parallel$ P1 $\parallel$ P2 $\parallel$ Lc $\parallel$ Le) and 2 B status word per response APDU (SW1 $\parallel$ SW2, appended by Android HCE automatically). (3) NFC physical-layer overhead—PCB octets, CRC, and SOF/EOF framing—is not counted, so the figures represent an upper bound on useful payload at the ISO-DEP layer [6]. (4) The LOADCERT payload size is an estimate; the actual value varies $\pm 10\%$ with the X.500 Distinguished Name lengths in the Reader certificate. (5) The AES-GCM IV (12 B) is derived from a counter and is never transmitted on the wire, so it does not contribute to link bytes.

A notable source of per-field overhead is the BER-TLV long-form length encoding. Any field whose encoded length exceeds 255 B requires a three-byte length header (0x82 $\parallel$ *len-hi* $\parallel$ *len-lo*) instead of one byte, contributing 4 B of tag-length overhead per field. Since every PQC object—Kyber public keys (≥ 800 B), ciphertexts (≥ 768 B), and Dilithium public keys (≥ 1312 B) and signatures (≥ 2420 B)—exceeds this threshold, each such field incurs this penalty.

Default configuration: ML-DSA-65 $\times$ ML-KEM-768. Table 3 breaks down the communication overhead for the default parameter set (Dilithium3 $\times$ Kyber768, NIST Security Level 3 [25,26]) across the six logical message exchanges of the Expedited flow.

Comparison with ECC Aliro. The classical ECC Aliro flow (Curve P-256) transfers approximately 1.2 KB (comprising an ephemeral ECDH public key 65 B, an ECDSA signature approximately 72 B, a P-256 certificate approximately 1 000 B, and framing overhead across approximately 8 APDUs, totalling approximately 1.2 KB) per transaction: an EC ephemeral public key of 65 B, an ECDSA signature of roughly 72 B, and the long-term certificate. PQ-Aliro (D3 $\times$ K768) incurs approximately $16.4/1.2 \approx 14\times$ more link bytes. The dominant contributors are the Dilithium3 public key (1,952 B, approximately $30\times$ the 65 B EC key) and the

Table 4. Communication overhead for all nine PQC parameter combinations (with LOADCERT). Security levels follow NIST PQC standards [25,26]. The “With key-slot” column shows the reduced overhead when LOADCERT is eliminated by pre-caching the Reader certificate on the User Device (saving approximately 4.0 KB for Level 2, approximately 5.5 KB for Level 3, and approximately 7.5 KB for Level 5).

Configuration	NIST Level	Total APDUs	Total bytes	With key-slot
ML-DSA-44 × ML-KEM-512	2	55	approximately 11.9 KB	approximately 7.9 KB
ML-DSA-44 × ML-KEM-768	2	58	approximately 12.6 KB	approximately 8.6 KB
ML-DSA-44 × ML-KEM-1024	2	62	approximately 13.5 KB	approximately 9.5 KB
ML-DSA-65 × ML-KEM-512	3	72	approximately 15.8 KB	approximately 10.3 KB
ML-DSA-65 × ML-KEM-768	3	75	≈16.4 KB	≈11.2 KB
ML-DSA-65 × ML-KEM-1024	3	79	approximately 17.3 KB	approximately 11.8 KB
ML-DSA-87 × ML-KEM-512	5	95	approximately 21.6 KB	approximately 14.1 KB
ML-DSA-87 × ML-KEM-768	5	98	approximately 22.3 KB	approximately 14.8 KB
ML-DSA-87 × ML-KEM-1024	5	102	approximately 23.2 KB	approximately 15.7 KB

Dilithium3 signature (3,309 B, approximately $46\times$ the 72 B ECDSA signature); the Kyber768 ciphertext (1,088 B) is approximately $17\times$ larger than an ECDH ephemeral point. The single largest contributing phase is LOADCERT, which alone accounts for approximately 5.5 KB (approximately 33% of the total), as it must transmit the Reader’s complete Dilithium certificate.

LOADCERT elimination. When the User Device already caches the Reader’s certificate (via a **key-slot** index or pre-provisioning), the LOADCERT exchange can be skipped entirely, saving approximately 22 APDUs and 5.5 KB. Total overhead drops to approximately 52 APDUs / 11.2 KB—a 32% reduction that largely closes the gap with the D2 configurations while maintaining NIST Level 3 security.

All parameter combinations. Table 4 summarises the nine ML-DSA/ML-KEM parameter combinations. Within each Dilithium tier, Kyber variants account for a modest 0.7–1.7 KB spread; crossing Dilithium tiers (D2 $\rightarrow$ D3 $\rightarrow$ D5) introduces 4–5 KB jumps, driven entirely by the growth in Dilithium public key and signature sizes [26].

8.2 NFC Transmission Latency

Note. All latency figures in this subsection are theoretical estimates derived from link-byte counts and published NFC bit-rate specifications; no physical NFC testbed measurement was performed. Figures represent lower bounds on total transaction time; see Section 9.2 for the scope of empirical validation.

Physical-layer model. ISO/IEC 14443 Type A supports four bit rates: 106, 212, 424, and 848 kbit/s. At 106 kbit/s, block-level overhead and turnaround delays reduce the effective application throughput to approximately 8 kB/s; higher rates scale quasi-linearly with modest additional framing costs [6]. Android HCE typically negotiates 424 kbit/s with compatible readers, making this the expected operating point for smartphone-based access control deployments.

Table 5. Estimated NFC transmission time (link bytes only, excluding cryptographic computation). Values are theoretical estimates; physical CRC/PCB framing is not included.

Bit rate	D3×K768 (16.4 KB)	D2×K512 (11.9 KB)	D5×K1024 (23.2 KB)
106 kbit/s	approximately 2.0 s	approximately 1.5 s	approximately 2.9 s
424 kbit/s	approximately 0.5 s	approximately 0.4 s	approximately 0.7 s
848 kbit/s	approximately 0.25 s	approximately 0.20 s	approximately 0.36 s

Methodology and scope of estimates. The latency figures in Table 5 account for bit-transmission time (total link bytes divided by bit rate) only. Per-APDU round-trip overhead—including ISO/IEC 14443-4 [17] Frame Waiting Time (FWT) and Reader polling intervals—is not included; for a 75-APDU exchange these round-trip costs can add several hundred milliseconds on real hardware. Additionally, the implementation’s asynchronous signing mechanism introduces a fixed 500 ms polling interval per signing cycle (one cycle per AUTH1 Command), contributing an additional ≈ 0.5 s to the real-device latency. Reported figures should therefore be treated as lower bounds on total transaction time.

Transmission time estimates. Table 5 reports estimated NFC transmission times for three representative parameter sets at three bit rates. The values reflect APDU link bytes only and exclude Reader-side and UD-side cryptographic computation times.

Cryptographic computation overhead. The transmission figures above must be augmented with on-device computation time. On a mid-range Android device, ML-DSA-65 signature generation requires approximately 50–100 ms and signature verification approximately 5 ms, consistent with results reported for the Kyber/Dilithium family on Android hardware [34,1]. ML-KEM encapsulation and decapsulation each contribute a further < 5 ms. Combining transmission and computation, the total expected latency for the default D3×K768 configuration at 424 kbit/s falls in the range of **0.6–1.5 s**.

User experience impact. By comparison, classical ECC Aliro transfers approximately 1.2 KB and completes in roughly 0.2 s end-to-end at 424 kbit/s. PQ-Aliro therefore introduces a $3\text{--}7\times$ latency increase. At the lower end (0.6 s), the additional delay is likely imperceptible during a normal door-tap gesture; at the upper end (1.5 s), it approaches the user-experience threshold for access-control interactions, motivating the LOADCERT-elimination optimisation described in Section 8.1. We note that all values in Table 5 are theoretical estimates; empirical measurement on physical NFC hardware constitutes important future work.

8.3 Security Level vs. Overhead Trade-off

The nine configurations in Table 4 span three NIST security levels and a $1.95\times$ overhead range (11.9 KB to 23.2 KB). A Pareto analysis across the security–overhead space identifies the following deployment profiles.

NIST Level 2 (D2 configurations, 11.9–13.5 KB). The D2×K512 configuration achieves the minimum overhead of approximately 11.9 KB—a 27% reduction relative to the default—while still meeting the NIST Level 2 security target [25,26]. This profile is suitable for deployments where transmission latency is critical (e.g., high-throughput turnstiles) and where the reduced margin against future cryptanalytic advances is acceptable within the organisation’s threat model.

NIST Level 3 (D3 configurations, 15.8–17.3 KB). The D3×K768 configuration represents the Pareto-optimal balance between security and overhead. It provides NIST Security Level 3 (AES-192 equivalent, approximately 96-bit post-quantum security) for both the key-encapsulation and signature primitives, and its approximately 16.4 KB footprint is manageable at 424 kbit/s. We adopt this as the default for PQ-Aliro [25,26].

NIST Level 5 (D5 configurations, 21.6–23.2 KB). The D5×K1024 configuration offers the highest available security (256-bit post-quantum), at a cost of +41% overhead relative to the default. With an estimated 2.9 s transmission time at 106 kbit/s, this profile is impractical at the slowest NFC speed but fully viable at 424 kbit/s (approximately 0.7 s) and, assuming the theoretical latency estimates hold on real hardware, would be suitable for high-assurance installations such as data-centre access control or government facilities.

Summary. Deployers can select the parameter set that best matches their threat model: D2×K512 for latency-sensitive or resource-constrained environments, D3×K768 as a balanced default, and D5×K1024 for maximum long-term security. The **key-slot** optimisation (Section 8.1) orthogonally reduces overhead by approximately 5.5 KB across all tiers, and should be enabled whenever the deployment architecture permits Reader-certificate pre-provisioning on the User Device.

The overhead figures above, together with the latency estimates, motivate the optimisation strategies and deployment trade-offs discussed in the following section.

9 Discussion and Optimization

9.1 Optimization Strategies

The default PQ-Aliro configuration (ML-DSA-65 × ML-KEM-768) incurs approximately 16.4 KB across 75 APDU exchanges—roughly 14× the overhead of a classical ECC-based Aliro flow. We identify three complementary strategies that can substantially reduce this cost.

LOADCERT Elimination via key-slot. The Aliro 1.0 specification [6] defines a **key-slot** field that allows a User Device (UD) to reference a Reader certificate already stored in its credential store, bypassing the explicit LOADCERT exchange. Eliminating LOADCERT removes approximately 22 APDU exchanges

and saves ~ 5.5 KB, reducing the total footprint to ~ 11.2 KB across 52 APDUs—a 32% reduction. This strategy is well-suited to enterprise deployments where a fixed set of Readers is trusted and their Dilithium certificates can be pre-provisioned at enrollment time. The trade-off is operational: it requires an out-of-band certificate provisioning infrastructure and complicates Reader certificate rotation. Nevertheless, for high-frequency access scenarios such as corporate office doors, the latency benefit of skipping a 22-round sub-flow is significant.

Downgrading to NIST Security Level 2. FIPS 203 [25] and FIPS 204 [26] both specify parameter sets at Security Levels 2, 3, and 5. Switching from the Level-3 combination (ML-DSA-65 $\times$ ML-KEM-768) to the Level-2 combination (ML-DSA-44 $\times$ ML-KEM-512) yields a total overhead of ~ 11.9 KB—a 27% saving over the default. This configuration is appropriate for lower-risk deployments (e.g., gym or library access) where the quantum threat horizon is more distant, but it is unsuitable for high-assurance use cases such as data-centre entry or government facilities, where Level-3 or Level-5 protection is warranted.

Profile0000 Certificate Compression. The Aliro Profile0000 certificate uses a custom DER encoding that includes full Distinguished Name (DN) fields totalling approximately 200 bytes. In scenarios where a `key-slot` index is used in the `AUTH1 Response` instead of transmitting the UD’s Dilithium public key inline, an additional ~ 1.9 KB can be saved. Further compression of the DN fields or adoption of a compact certificate format would provide incremental gains without compromising the authentication semantics defined in the Aliro specification [6].

9.2 Limitations

Several important caveats apply to the results presented in this paper.

Theoretical byte counts only. All communication overhead figures are derived from static APDU-layer byte accounting and do not include ISO/IEC 14443 physical-layer framing (PCB bytes, CRC-16, SOF/EOF delimiters). Actual over-the-air data volumes will be marginally higher; the relative comparisons between configurations remain valid.

No real NFC hardware. End-to-end protocol execution was validated in a software emulation environment: a PC-based Reader process communicating with an Android Emulator running the HCE stack. Real NFC hardware introduces RF field establishment latency, reader polling intervals, and device-specific RF stack overhead that are not captured by emulation. Latency figures for real deployments may differ from our estimates.

Estimated cryptographic timing. The ML-DSA-65 signing time of 50–100 ms cited in Section 8 is extrapolated from published benchmarks on comparable ARM processors; it has not been measured directly on the test device. Performance will vary with JVM warm-up state, concurrent background processes, and the specific Android version and security chip configuration of the target handset.

Certificate pre-provisioning complexity. The LOADCERT-elimination optimisation assumes that the Credential Issuer can reliably pre-provision Reader

certificates into each UD at enrollment time and keep them synchronised across certificate renewals. This requirement adds non-trivial infrastructure burden and may not be feasible in open-ended deployments where any compliant Reader should be able to interact with any UD without prior pairing.

Certificate revocation in key-slot mode. The `key-slot` optimisation pre-provisions Reader certificates on the User Device. No revocation mechanism is specified: if a Reader device is compromised or decommissioned, User Devices that have cached its certificate will continue to authenticate to it. Incorporating certificate status checks (e.g., OCSP stapling or short-lived certificates) within the NFC session would add overhead but is essential for enterprise deployments where Reader devices may be revoked.

Reader-side linkability. Our tracking-resilience theorem (Section 5) captures User Device identity privacy only; reader-side identity is explicitly outside the model’s scope, because the `reader-identifier` is transmitted in plaintext in the AUTH0 Command by design in the Aliro 1.0 specification [6]. The `reader-identifier` field (32 bytes) is transmitted in plaintext within the AUTH0 Command, allowing a passive eavesdropper to correlate all NFC transactions at a fixed reader location. While the Aliro 1.0 specification [6] transmits this field unencrypted by design, it constitutes a form of reader-side linkability that our tracking-resilience model (Section 4) does not capture. Future work may consider deriving a session-specific reader pseudonym from the `transaction-identifier` to mitigate this leakage.

Relay attack exposure. PQ-Aliro, like all NFC-based access-control protocols, relies on the ISO/IEC 14443 [17] physical-layer range limit (≈ 10 cm) rather than cryptographic distance-bounding to prevent relay attacks. A relay adversary who tunnels the NFC channel over a longer distance can open a door without the legitimate user’s knowledge. The `transaction-identifier` provides replay protection but not relay protection. Integrating distance-bounding mechanisms [6] or UWB ranging (available in the Aliro specification) would address this threat.

Single transport protocol. The current implementation and analysis cover NFC (ISO/IEC 14443) exclusively. The Aliro specification also defines BLE and UWB transport paths, each with distinct frame-size constraints and session-key derivation flows. The overhead impact of PQC on those transports is an independent open question.

9.3 Future Work

Several directions extend the contributions of this paper.

Real hardware evaluation. Measuring end-to-end protocol latency on commercial NFC-enabled smartphones and readers—including RF field setup time and Reader-side processing—would validate our theoretical estimates and surface implementation-specific bottlenecks.

Falcon-512 integration. Implementing and benchmarking a Falcon-512 (FIPS 206) variant of PQ-Aliro on representative Android devices would deter-

mine whether the substantial signature-size advantage translates into acceptable key-generation latency in practice.

BLE and UWB transports. Extending PQC integration to the Aliro BLE and UWB flows—particularly the derivation of BleSK and URSK session keys—would provide a complete picture of quantum-safe Aliro across all supported transports.

Interoperability with CCC Digital Key 3.0. Exploring cross-specification interoperability with the CCC Digital Key 3.0 ecosystem [5] would clarify whether a unified PQC upgrade path is achievable across both standards, given their overlapping deployment contexts.

Key-slot provisioning protocol. Designing and formalising a certificate pre-provisioning protocol that enables LOADCERT elimination at scale—with support for certificate rotation and revocation—would make this optimisation deployable in real enterprise access-control infrastructures. The core formalisation challenge is modelling pre-provisioned Reader certificates within the AKE security game: the LOADCERT round must be replaced by a certificate-registration oracle capturing both correctness (the UD accepts only certificates from authorised issuers) and privacy (pre-provisioned certificates do not leak Reader-UD pairing history). Certificate rotation and revocation in this model remain open problems.

10 Conclusion

This paper presented PQ-Aliro, the first complete post-quantum variant of the Aliro NFC access-control protocol [6]. Our design preserves the three-round Expedited Flow (AUTH0/LOADCERT/AUTH1), replacing the classical primitives with NIST-standardised counterparts: ECDH is superseded by ML-KEM-768 and ECDSA by ML-DSA-65 [25,26], while session confidentiality is maintained by HKDF (HMAC-SHA256 + HKDF-Expand) and AES-256-GCM. Within the AKE security model of Schäge et al. [30], we prove that PQ-Aliro achieves mutual authentication, forward secrecy (under long-term key corruption), and identity privacy, with the security bound reducing to the IND-1CCA security of ML-KEM-768, the EUF-CMA security of ML-DSA-65, the PRF security of HKDF, and the AEAD security of AES-256-GCM. The use of IND-1CCA—a weaker assumption than IND-CCA2—is sufficient because PQ-Aliro performs exactly one KEM encapsulation per session, yielding a tighter security reduction than a full CCA2-based proof would provide. A full open-source implementation targeting Android HCE and a PC NFC Reader was developed using BouncyCastle 1.78 BCPQC, and an end-to-end test suite comprising 20 assertions across nine PQC configurations passed without exception. Key engineering findings—Dilithium private-key isolation inside a dedicated Android named process with immediate self-termination, and asynchronous ML-DSA signing to prevent HCE main-thread deadlock—are documented for the benefit of practitioners porting lattice-based protocols to Android HCE environments.

The principal cost of this upgrade is communication overhead. The default D3×K768 configuration exchanges approximately 16.4 KB across 75 APDUs, roughly 14× the footprint of the original ECC-based flow (≈ 1.2 KB), with the bulk of the expansion attributable to the larger public keys and signatures of ML-DSA-65. Enabling the `key-slot` optimisation—which suppresses the LOAD-CERT round once a certificate is cached on the reader—reduces this to ≈ 11.2 KB, a 32% saving. Alternatively, the lighter D2×K512 configuration reaches ≈ 11.9 KB at NIST Security Level 2. At 424 kbit/s NFC bit-rate, the projected end-to-end latency of 0.6–1.5 s remains acceptable for the vast majority of physical-access scenarios.

With NIST IR 8547 [27] mandating migration to post-quantum algorithms by 2035, the urgency of hardening NFC access-control systems against harvest-now/decrypt-later attacks cannot be overstated. PQ-Aliro provides a concrete, standards-compliant migration path for deployments built on the Aliro [6] ecosystem. Future work will focus on empirical latency measurements on production NFC hardware, evaluation of Falcon-512 as a lighter signature alternative, and extension of the PQ-Aliro handshake to BLE and UWB transport channels already defined in the Aliro specification.

References

1. Becker, H., Hwang, V., Kannwischer, M.J., Yang, B.Y., Yang, S.Y.: Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2022**(1), 235–263 (2022). <https://doi.org/10.46586/tches.v2022.i1.235-263>
2. Bellare, M., Rogaway, P.: Entity authentication and key distribution. In: Stinson, D.R. (ed.) *Advances in Cryptology – CRYPTO’93. Lecture Notes in Computer Science*, vol. 773, pp. 232–249. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 22–26, 1994). https://doi.org/10.1007/3-540-48329-2_21
3. Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy (EuroS&P). pp. 353–367 (2018). <https://doi.org/10.1109/EuroSP.2018.00032>
4. Canetti, R., Krawczyk, H.: Analysis of key-exchange protocols and their use for building secure channels. In: Pfitzmann, B. (ed.) *Advances in Cryptology – EUROCRYPT 2001. Lecture Notes in Computer Science*, vol. 2045, pp. 453–474. Springer, Heidelberg, Germany, Innsbruck, Austria (May 6–10, 2001). https://doi.org/10.1007/3-540-44987-6_28
5. Car Connectivity Consortium: CCC digital key release 3.0. Tech. rep., Car Connectivity Consortium (2021), <https://carconnectivity.org/digital-key-specification-download-2/>
6. Connectivity Standards Alliance: Aliro 1.0 specification. Tech. Rep. 26-42802-001, Connectivity Standards Alliance (2026), <https://csa-iot.org/all-solutions/aliro/>
7. Dowling, B., Fischlin, M., Günther, F., Stebila, D.: A cryptographic analysis of the TLS 1.3 handshake protocol. *Journal of Cryptology* **34**(4), 37 (Oct 2021). <https://doi.org/10.1007/s00145-021-09384-1>

8. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2018**(1), 238–268 (2018). <https://doi.org/10.13154/tches.v2018.i1.238-268>
9. Dworkin, M.: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. Special Publication 800-38D, National Institute of Standards and Technology (2007). <https://doi.org/10.6028/NIST.SP.800-38D>
10. Fischlin, M., Günther, F.: Multi-stage key exchange and the case of Google’s QUIC protocol. In: Ahn, G.J., Yung, M., Li, N. (eds.) *ACM CCS 2014: 21st Conference on Computer and Communications Security*. pp. 1193–1204. ACM Press, Scottsdale, AZ, USA (Nov 3–7, 2014). <https://doi.org/10.1145/2660267.2660308>
11. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Strongly secure authenticated key exchange from factoring, codes, and lattices. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) *PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography. Lecture Notes in Computer Science*, vol. 7293, pp. 467–484. Springer, Heidelberg, Germany, Darmstadt, Germany (May 21–23, 2012). https://doi.org/10.1007/978-3-642-30057-8_28
12. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Practical and post-quantum authenticated key exchange from one-way secure key encapsulation mechanism. In: Chen, K., Xie, Q., Qiu, W., Li, N., Tzeng, W.G. (eds.) *ASIACCS 13: 8th ACM Symposium on Information, Computer and Communications Security*. pp. 83–94. ACM Press, Hangzhou, China (May 8–10, 2013)
13. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology* **26**(1), 80–101 (2013). <https://doi.org/10.1007/s00145-011-9114-1>
14. Goldwasser, S., Micali, S., Rivest, R.L.: A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing* **17**(2), 281–308 (Apr 1988)
15. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. pp. 212–219 (1996). <https://doi.org/10.1145/237814.237866>
16. Hülsing, A., Ning, K.C., Schwabe, P., Weber, F., Zimmermann, P.R.: Post-quantum WireGuard. In: *2021 IEEE Symposium on Security and Privacy*. pp. 304–321. IEEE Computer Society Press, San Francisco, CA, USA (May 24–27, 2021). <https://doi.org/10.1109/SP40001.2021.00030>
17. Identification cards – Contactless integrated circuit cards – Proximity cards – Part 4: Transmission protocol (2018)
18. Krawczyk, H., Eronen, P.: HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, IETF (2010), <https://www.rfc-editor.org/rfc/rfc5869>
19. Krawczyk, H.: HMQV: A high-performance secure Diffie-Hellman protocol. In: Shoup, V. (ed.) *Advances in Cryptology – CRYPTO 2005. Lecture Notes in Computer Science*, vol. 3621, pp. 546–566. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 14–18, 2005). https://doi.org/10.1007/11535218_33
20. Krawczyk, H., Bellare, M., Canetti, R.: HMAC: keyed-hashing for message authentication. RFC 2104, 1–11 (1997). <https://doi.org/10.17487/RFC2104>, <https://doi.org/10.17487/RFC2104>
21. LaMacchia, B.A., Lauter, K., Mityagin, A.: Stronger security of authenticated key exchange. In: Susilo, W., Liu, J.K., Mu, Y. (eds.) *ProvSec 2007: 1st International*

- Conference on Provable Security. Lecture Notes in Computer Science, vol. 4784, pp. 1–16. Springer, Heidelberg, Germany, Wollongong, Australia (Nov 1–2, 2007)
22. Li, Y., Schäge, S.: No-match attacks and robust partnering definitions: Defining trivial attacks for security protocols is not trivial. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017: 24th Conference on Computer and Communications Security. pp. 1343–1360. ACM Press, Dallas, TX, USA (Oct 31 – Nov 2, 2017). <https://doi.org/10.1145/3133956.3134006>
 23. Lyu, Y., Liu, S., Han, S., Gu, D.: Privacy-preserving authenticated key exchange in the standard model. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology – ASIACRYPT 2022, Part III. Lecture Notes in Computer Science, vol. 13793, pp. 210–240. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). https://doi.org/10.1007/978-3-031-22969-5_8
 24. National Institute of Standards and Technology: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Federal Information Processing Standards Publication 202, NIST (2015). <https://doi.org/10.6028/NIST.FIPS.202>
 25. National Institute of Standards and Technology: FIPS 203: Module-lattice-based key-encapsulation mechanism standard. Federal information processing standard, NIST (2024). <https://doi.org/10.6028/NIST.FIPS.203>
 26. National Institute of Standards and Technology: FIPS 204: Module-lattice-based digital signature standard. Federal information processing standard, NIST (2024). <https://doi.org/10.6028/NIST.FIPS.204>
 27. National Institute of Standards and Technology: Transition to post-quantum cryptography standards. Tech. Rep. NIST IR 8547 (Initial Public Draft), NIST (2024), <https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf>
 28. Pan, J., Wagner, B., Zeng, R.: Lattice-based authenticated key exchange with tight security. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023, Part V. Lecture Notes in Computer Science, vol. 14085, pp. 616–647. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38554-4_20
 29. Rogaway, P., Shrimpton, T.: A provable-security treatment of the key-wrap problem. In: Vaudenay, S. (ed.) Advances in Cryptology – EUROCRYPT 2006. Lecture Notes in Computer Science, vol. 4004, pp. 373–390. Springer Berlin Heidelberg, Germany, St. Petersburg, Russia (May 28 – Jun 1, 2006). https://doi.org/10.1007/11761679_23
 30. Schäge, S., Schwenk, J., Lauer, S.: Privacy-preserving authenticated key exchange and the case of IKEv2. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol. 12111, pp. 567–596. Springer, Cham, Switzerland, Edinburgh, UK (May 4–7, 2020). https://doi.org/10.1007/978-3-030-45388-6_20
 31. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020: 27th Conference on Computer and Communications Security. pp. 1461–1480. ACM Press, Virtual Event, USA (Nov 9–13, 2020). <https://doi.org/10.1145/3372297.3423350>
 32. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Journal on Computing **26**(5), 1484–1509 (1997). <https://doi.org/10.1137/S0097539795293172>
 33. Shoup, V.: A proposal for an ISO standard for public key encryption. Cryptology ePrint Archive, Report 2001/112 (2001), <https://eprint.iacr.org/2001/112>

34. da Silva Lima, J.P., Ribeiro, L.A.D.S., de Queiroz, R.J.G.B., Quintino, J.P., da Silva, F.Q.B., Santos, A.L.M.: Evaluating Kyber post-quantum KEM in a mobile application. In: NIST Third Post-Quantum Cryptography Standardization Conference (2021), <https://csrc.nist.gov/CSRC/media/Events/third-pqc-standardization-conference/documents/accepted-papers/ribeiro-evaluating-kyber-pqc2021.pdf>